

Addressed Parameter Memories: A High-Level Perspective on Continual Learning in Neural Systems

Daniel MacDonald

July 3, 2026

Abstract

We sketch a high-level approach to continual learning in neural systems based on addressed parameter memories. The central idea is that a continually learning system should not be viewed as maintaining a single mutable parameter vector, but as maintaining an addressable graph of parameter states, deltas, residual modules, and compressed macro-nodes. At training and inference time, the system pays an addressing cost to recover or compose the parameterization appropriate to the current situation. This perspective is motivated by resource-bounded incremental induction principles, by the view that recall and adaptation require search over compressed histories, and by intuitions from incremental compression systems such as IC/WILLIAM, where local resource-bounded rewrites improve a compression DAG without global recomputation. Predictive coding gives a convenient concrete instantiation, but the perspective is not essentially tied to predictive coding: any energy-based or variational model with a tractable objective over latent states and parameter addresses can serve the same role. We show that ordinary stochastic gradient descent and mixture-of-experts architectures arise as limiting cases in different dimensions, and that many standard continual-learning and parameter-efficient-learning “tricks” are natural approximations to the same core mechanism. We then compare this view to Goertzel’s proposed columnar predictive-coding residual architecture and argue that many of its continual-learning properties follow naturally from sparse addressed residual memories, while exact recall requires an additional archival or parameter-DAG mechanism inside the columns.

1 Motivation: Continual Learning as Resource-Bounded Addressing

The starting point is the RBII view of continual learning: an online learner is a resource-bounded system that must continually trade off present performance, retention of prior competence, transfer to future regimes, memory cost, compute cost, and the cost of finding the right stored structure when it is needed [MacDonald, 2026a,b]. In this view, recall is not free. A system may have retained enough information to recover an old competence, but recovering it may require search through a tree, graph, cache, or compressed model of prior updates. The relevant cost is not only the number of stored bits, but the addressing cost required to bring the right bits into effective control.

This suggests a reframing of catastrophic forgetting. Standard neural continual learning typically asks how a single parameter vector can be updated without destroying previous performance. The addressed-memory view instead asks how an online learner can maintain a library or graph of parameter states and parameter deltas such that old competences remain recoverable at bounded cost. A system may retain exact parameter leaves, compressed approximations to leaves, shared delta atoms, task-local residuals, or generalized macro-nodes. Forgetting occurs when a previously

useful state is no longer reachable, is only reachable at excessive addressing cost, or has been lossy-compressed beyond the tolerance required by the task.

This is also where the connection to IC/WILLIAM enters. In incremental compression, one can view learning as the construction and local refinement of a compression DAG. The system need not globally recompress its entire history whenever new data arrive. Instead, it can perform local, resource-bounded rewrites that reduce description length, introduce reusable intermediate nodes, or replace families of residuals by shorter shared structure. The proposal here borrows that intuition and applies it to neural parameter space: parameter states reached during online learning are treated as targets in a compression problem, and the learner searches for a graph of deltas and macro-deltas that reconstructs or approximates those states at low description length and acceptable addressing cost.

The exact-recall version of the idea is only a starting point. If every historical parameter state is stored exactly, no forgetting is necessary, but memory and search cost grow without bound. Practical systems must relax exact recall, consolidating old leaves, averaging or merging compatible deltas, dropping small residuals, distilling parameter states into hypernetworks, or anchoring compression through replay or generative resampling. Generalization is not assumed merely because the system is neural; it is purchased by compression tactics. When many leaves can be reconstructed from shared parents, low-rank deltas, common residual modules, or compact generators, the system has found reusable structure.

Predictive coding is a useful concrete language for this perspective because it already treats inference as energy minimization over latent states, and it softens the separation between inference and learning [Rao and Ballard, 1999, Whittington and Bogacz, 2017]. However, predictive coding is not essential. A more general energy-based model is sufficient as long as we have an energy or free-energy functional over observations, latent states, and parameter addresses, together with a tractable method for minimizing or approximately minimizing that functional under resource constraints [LeCun et al., 2006].

2 Core Formulation

Let x denote an observation or partially observed task instance, z latent variables, and θ the parameters of a neural model. In a predictive-coding generative model one might write

$$x \leftarrow f_{\theta}(z),$$

with energy

$$\mathcal{E}(x, z, \theta) = \frac{1}{2} \sum_{\ell} \|z_{\ell-1} - f_{\ell}(z_{\ell}; \theta_{\ell})\|_{\Sigma_{\ell}^{-1}}^2 + R_z(z).$$

More generally, $\mathcal{E}$ may be any energy, loss, variational free energy, or task score that supports approximate inference over z .

The addressed-parameter-memory move is to replace the single live parameter vector by an expression over a graph of parameter atoms. Let

$$G_t = (V_t, E_t)$$

be a parameter-memory graph at time t . It contains a root parameter vector θ_0 , frozen delta atoms Δ_j , and expression codes c_v associated with nodes v . Evaluating a code gives an effective parameter vector

$$\theta_G(c_v) = \theta_0 + \sum_j c_{v,j} \Delta_j.$$

At inference and training time, the system introduces an address a as an additional inferred variable:

$$\theta_{\text{eff}}(a) = \theta_G(a) = \theta_0 + \sum_j a_j \Delta_j.$$

The online inference problem is then

$$(z, a)^{\text{arg min}_{z, a} [\mathcal{E}(x, z, \theta_G(a)) + \lambda_{\text{addr}} R_G(a)]}.$$

The graph prior $R_G(a)$ encodes the cost of addressing. It may penalize long paths, high-entropy mixtures, dense coefficient vectors, low-use branches, cache misses, nonlocal jumps, or any other feature that makes recall expensive. In a tree version, a is close to a path or prefix. In a DAG version, a may select and combine multiple delta atoms or macro-nodes.

Thus the central object is

$$\theta_{\text{eff}}(x) = \theta_0 + \sum_j a_j(x) \Delta_j,$$

where $a(x)$ is not merely a learned feed-forward router, but an inferred or amortized address into a growing memory of parameter structure.

3 Online Training

For each incoming example x_t , the system performs addressed inference, local update, and graph maintenance.

The sparsity and entanglement penalties serve two functions. First, they reduce the description length of each new delta. Second, they bias updates toward directions that are less likely to interfere with already stored branches. The parameter vector after training is therefore not merely a vector; it is an expression over a dependency graph of frozen or partially frozen update atoms.

4 Inference and Recall

At inference time, the system does not assume that one global parameter vector is optimal. It searches for, or amortizes the search for, a suitable parameter expression.

This gives a clean interpretation of fast and slow recall. Cheap content indices and coarse macro-nodes provide fast focus. More expensive graph search, local energy minimization, or retrieval from deeper branches provides slower but more accurate recall. Addressing cost is the extra compute required to recover a better parameter expression.

5 Exact Recall as a Starting Point

The strongest version of the framework gives an exact-recall guarantee.

Proposition 1 (Exact reachable-state recall). *Suppose that after training on x_i , the system stored an address c_i such that $\theta_G(c_i) = \theta_i^+$, where θ_i^+ was the parameter state achieved for x_i . If c_i remains reachable at future time t , then future minimum energy on x_i is no worse than the energy achieved when x_i was stored.*

Algorithm 1 Online Addressed-Parameter Training

- 1: Observe x_t .
- 2: Retrieve or propose candidate graph regions using content indices, recent states, latent keys, error signatures, or coarse macro-nodes.
- 3: Solve the addressed inference problem

$$(z_t^{a_t})^{\arg \min_{z,a} \mathcal{E}(x_t, z, \theta_G(a)) + \lambda_{\text{addr}} R_G(a)}.$$

- 4: Let $\hat{\theta}_t = \theta_G(a_t)$.
- 5: Compute a local update direction from the energy gradient:

$$g_t = \nabla_{\theta} \mathcal{E}(x_t, z_t^{\hat{\theta}_t}).$$

- 6: Form a sparse or low-description-length delta:

$$\Delta_t = \arg \min_{\Delta} \left[\mathcal{E}(x_t, z_t^{\hat{\theta}_t + \Delta}) + \lambda_{\Delta} \text{DL}(\Delta) + \lambda_{\text{ent}} \Omega(\Delta; G_t) + \frac{1}{2\eta} \|\Delta\|^2 \right].$$

- 7: Optionally approximate this by

$$\Delta_t \approx -\eta \text{SparseProject}(g_t).$$

- 8: Store a new node or edge:

$$c_t = a_t^{+b_t e_t},$$

where e_t selects the new frozen delta atom and b_t may be chosen by line search.

- 9: Record metadata: content key, latent state, error signature, achieved energy, parent path, coefficient magnitude, usage count, and consolidation priority.
 - 10: Run bounded background graph maintenance when memory, search cost, or redundancy thresholds are exceeded.
-

Proof. Since c_i remains in the feasible address set,

$$\min_{z,a} \mathcal{E}(x_i, z, \theta_G(a)) \leq \min_z \mathcal{E}(x_i, z, \theta_G(c_i)).$$

But $\theta_G(c_i) = \theta_i^+$, so the old settled state remains available. Therefore the minimum achievable energy is no worse than the previously achieved energy. $\square$

This guarantee is not meant as the final practical design. Exact recall by leaf retention is memory-expensive and can make addressing costly. Its value is conceptual: it separates forgetting into distinct mechanisms. Forgetting occurs when a leaf is deleted, when a lossy consolidation changes its represented function, when search cannot find the relevant address under resource bounds, or when the energy being optimized is misaligned with the task-relevant notion of recall.

In practice we relax exact recall. The useful engineering problem is to trade off memory, addressing cost, and tolerated forgetting by consolidating graph regions while bounding or empirically measuring energy increases on retained examples, replay samples, or task probes.

Algorithm 2 Addressed Inference

- 1: Given query x , retrieve K candidate graph regions using content-linked indices, latent keys, task signatures, residual error signatures, or recent successful branches.
- 2: For each candidate region N_k , solve

$$(z_k^{a_k})^{\text{arg min}_{z, a \in N_k} \mathcal{E}(x, z, \theta_G(a)) + \lambda_{\text{addr}} R_G(a)}.$$

- 3: Select the best candidate by addressed free energy, or maintain a posterior

$$p(a, z \mid x) \propto \exp[-\mathcal{F}_G(x, z, a)].$$

- 4: Predict, reconstruct, or act using the selected or averaged effective parameters $\theta_G(a)$.
 - 5: If the fast solution is uncertain or high-energy, spend additional addressing cost on slower recall, broader graph search, or background consolidation.
-

6 Consolidation and Generalization

A lossless consolidation step replaces a graph G by a shorter graph G' while preserving selected parameter states:

$$G' = \arg \min_G \left[\text{DL}(G) + \sum_i \text{DL}(c_i) \right] \quad \text{s.t.} \quad \theta_G(c_i) = \theta_{v_i}.$$

A simple regional consolidation is

$$\bar{\theta}_R = \sum_{i \in R} w_i \theta_i, \quad r_i = \theta_i - \bar{\theta}_R.$$

If all residuals r_i are retained exactly, this is merely a refactoring. If residuals are dropped, quantized, approximated by existing deltas, or generated by a hypernetwork, the consolidation is lossy.

Generalization enters at the lossy or compressive stage. If many leaves can be expressed by short edits from a shared parent, then the shared parent captures a reusable regularity. If old deltas can be approximated by low-rank atoms, task vectors, columns, or hypernetwork outputs, then the graph has found structure in parameter space. If consolidation is supported by replay or generative resampling, the system can test whether a shorter parameter graph preserves behavior in data space rather than merely reconstructing weights.

This is why the framework does not require IID data in the way ordinary batch training does. IID sampling is one way to make averaging safe. Here, averaging is only one possible consolidation operator, and it is applied where the graph, energy, and probes indicate that the corresponding states are compatible.

7 Batch SGD as a Limiting Case

Ordinary batch SGD is recovered by collapsing all addresses into a single global parameter state. Given a batch B ,

$$\Delta_B = -\eta \frac{1}{|B|} \sum_{x_i \in B} \nabla_{\theta} \mathcal{E}(x_i, z_i, \theta),$$

and

$$\theta \leftarrow \theta + \Delta_B.$$

In addressed-graph language, SGD temporarily creates many example-specific update tendencies and immediately performs a lossy consolidation:

$$\{\Delta_1, \dots, \Delta_n\} \rightsquigarrow \bar{\Delta}.$$

The graph has one live node, no recoverable branches, no explicit addressing, and no archived parameter leaves. Online SGD is the harsher case of a single path through parameter space.

Thus

SGD = one-node parameter memory + global update averaging + no contextual address search.

The IID assumption is precisely what makes this extreme compression tolerable. If examples are IID samples from one distribution, then averaging gradients is a reasonable estimator. If the stream is non-IID, multimodal, adversarial, or task-shifted, immediate averaging can destroy recoverability of earlier states.

8 Mixture-of-Experts as a Shallow Addressing Case

Mixture-of-experts architectures occupy a different axis. A sparsely gated MoE computes

$$h_{l+1} = \sum_{e \in \text{TopK}(g(x))} g_e(x) f_e(h_l; \theta_e)$$

[Shazeer et al., 2017]. In addressed-parameter notation, this is

$$\theta_{\text{eff}}(x) = \theta_0 + \sum_{e \in \text{TopK}(x)} a_e(x) \Delta_e.$$

An MoE is therefore an addressable parameter library with a shallow, architecture-fixed graph:

$$\text{root} \rightarrow \{\text{expert}_1, \dots, \text{expert}_M\}.$$

The address is usually inferred by a learned router rather than by energy minimization over a historical parameter graph. The experts are usually parallel modules trained jointly, not archived parameter states produced by an online history.

Thus

MoE = many module addresses + learned contextual router + little or no historical exact-recall graph.

Batch SGD is the one-address global-average limit. MoE is the shallow horizontal routing limit. The full addressed-memory framework allows deep historical structure, explicit deltas, amortized routing, energy-based address refinement, and graph compression.

9 Standard Tricks as Approximation Heuristics

A virtue of the perspective is that many familiar hacks and tropes fall out as natural approximations to the same core mechanism.

Adapters and LoRA. Adapters and LoRA restrict the allowed delta basis to low-dimensional or low-rank subspaces [Houlsby et al., 2019, Hu et al., 2022]. In the addressed-memory view, they are cheap parameter-delta atoms:

$$\Delta_j \approx U_j V_j^\top.$$

Prefix tuning and prompt tuning. Soft prompts and prefixes can be viewed as activation-address deltas rather than weight deltas. They fit the same pattern if both z and a are inferred:

$$(z, a) = \arg \min_{z, a} \mathcal{E}(x, z, \theta(a)).$$

EWC, SI, and related regularizers. Quadratic regularizers such as EWC approximate the constraint that new updates should not destroy old reachable states [Kirkpatrick et al., 2017]. They do this without explicitly storing and searching the full parameter graph:

$$L_{\text{new}}(\theta) + \frac{\lambda}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}}).$$

GEM, A-GEM, and OGD. Gradient projection methods constrain new updates so that old losses do not increase, or so that new gradients avoid old task-sensitive directions [Lopez-Paz and Ranzato, 2017, Farajtabar et al., 2020]. These can be read as local edge constraints on the parameter graph.

Replay and generative replay. Replay anchors consolidation in data space rather than parameter space [Shin et al., 2017]. Instead of requiring exact reconstruction of old leaves, the system tests whether old behavior is preserved on stored or generated samples.

Progressive networks, PathNet, PackNet, and Piggyback. Parameter-isolation methods allocate columns, paths, masks, or subnetworks to protect old tasks [Rusu et al., 2016, Fernando et al., 2017, Mallya and Lazebnik, 2018, Mallya et al., 2018]. These are addressed parameter memories with mostly discrete coefficients:

$$a_j \in \{0, 1\}.$$

MAML and meta-learning. MAML learns a root parameter state from which many tasks are reachable by short updates [Finn et al., 2017]. In the present language, MAML learns one good compression center. The addressed-memory framework generalizes this by permitting many historical or consolidated centers and by searching over which one to adapt from.

Hypernetworks. A hypernetwork is a compressed generator of parameter states or deltas [Ha et al., 2017]:

$$\Delta_j = H_\psi(c_j).$$

This is the natural next compression layer once explicit storage of deltas becomes too expensive.

Model soups, task arithmetic, TIES, and Git Re-Basin. Weight averaging and task-vector arithmetic are graph consolidation operations [Wortsman et al., 2022, Ilharco et al., 2023, Yadav et al., 2023]. Git Re-Basin highlights a key obstacle: raw weight-space arithmetic is not invariant to permutation symmetries and basin structure [Ainsworth et al., 2023]. These methods are useful both as heuristics and as warnings.

RAG, kNN memories, and episodic retrieval. Retrieval-augmented systems are content indices over examples, text, or memories. In this framework they become address proposal mechanisms: they identify candidate regions of the parameter graph before expensive energy minimization.

Test-time adaptation. Test-time adaptation is an ephemeral fork. A temporary delta is created for the current context, used for prediction, and then either discarded or committed to the graph.

Knowledge editing. A targeted model edit is a sparse parameter delta. The addressed-memory question is whether it should be treated as a local edge, a reusable atom, or a noncommutative branch that should not be merged blindly.

Ensembling and Bayesian model averaging. Instead of selecting one address, the system can average over a posterior:

$$p(a, z \mid x) \propto \exp[-\mathcal{F}_G(x, z, a)].$$

Ensembling is posterior averaging over parameter addresses.

The unifying pattern is that these methods restrict, route among, protect, merge, retrieve, or compress parameter deltas. They differ mostly in which part of the addressed-memory problem they approximate.

10 Comparison to Goertzel’s Columnar Residual Architecture

Goertzel’s staged residual proposal places a structured residual system on top of a frozen base model:

$$F(x) = F_0(x) + R_\phi^{\text{cont}}(x) + R_\phi^{\text{sym}}(x)$$

[Goertzel, 2026]. The continuous component is trained with predictive-coding, causal-coding, weakness, and modularity objectives. The symbolic head projects hidden states into a template key space, retrieves symbolic templates from a MORK store, and integrates symbolic attention back into the residual stream.

At the columnar stage, the architecture becomes

$$F(x) = F_0(x) + \sum_{a \in S(x)} g_a(x) C_a(x) + R_\phi^{\text{sym}}(x),$$

where C_a are sparse residual columns, $S(x)$ is the active support, and $g_a(x)$ are gates. The columns may be composed from rank-one residual atoms:

$$C_a(x) = \sum_{i \in N(a)} q_{ai} A_i(x).$$

The distinctive additional mechanism is column-template fusion: each column has preferred symbolic templates, each template has preferred columns, and routing and retrieval bias one another.

In the addressed-memory view, the interpretation is direct:

$$C_a \approx \text{a reusable macro-delta or macro-node},$$

$$g_a(x) \approx \text{an amortized address coefficient},$$

and

$A_i \approx$ a low-rank or rank-one delta atom.

The symbolic head is a content-linked addressing system. Column-template fusion is a coupling between a continuous parameter-address space and a symbolic template-address space. A compact joint energy would be

$$\mathcal{F}(x, z, s, t) = \mathcal{E}(x, z, \theta(s)) + \lambda_s R_s(s) + \lambda_t R_t(t) - \beta s^\top B t + L_{\text{sym}}(t, h),$$

where s selects columns, t selects symbolic templates, and B stores column-template binding strengths.

Many of the continual-learning properties of the columnar residual architecture therefore derive naturally from the addressed-memory setup. Sparse routing reduces interference by localizing updates. Columns are macro-deltas that amortize search over frequently useful parameter regions. Rank-one atoms are cheap delta bases. Weakness, causal fingerprints, and information-geometric audits are local criteria for deciding which modules should be updated or consolidated. Symbolic retrieval gives a content-linked index into the residual ecology.

However, two additions are not forced by the minimal continual-learning framework.

First, explicit symbolic grounding is an additional architectural commitment. A generic addressed-memory system can use embeddings, latents, error signatures, or keys. Goertzel’s architecture wants stable linkage to Atomese/MORK/PLN-style symbolic structures. That requires symbolic templates, contrastive key-space alignment, template retrieval, and column-template binding. These are not necessary for continual learning per se, but they are central to neural-symbolic integration.

Second, exact recall is not automatic. Sparse columns and routers reduce interference, but if a column is mutable,

$$C_a \leftarrow C_a + \Delta,$$

then a new update can still damage an old context that routes through the same column. To inherit the stronger exact-recall property, each column must itself contain an addressed delta graph or archival mechanism:

$$C_a(x) = C_{a,0}(x) + \sum_j \alpha_{a,j}(x) \Delta_{a,j}(x).$$

Equivalently, the system needs archived leaf adapters, column-local branches, or recoverable expressions over column atoms. Without this addition, the columnar residual architecture is a strong anti-interference and transfer architecture, but not a mathematical exact-recall architecture.

11 Failure Modes

The following failure modes are structural rather than merely implementation-specific.

Ambiguous address. If the same observation requires different incompatible parameter states and no context variable distinguishes them, no addressing algorithm can choose the correct branch. The information is absent.

Incompressible streams. Exact no-forgetting can always be purchased by storing every leaf. Sublinear memory requires reusable structure. For adversarial or unrelated streams, there may be no shorter graph than the list of states itself.

Addressing cost dominates. The framework moves part of the continual-learning burden from parameter interference to search. If the graph grows and content indices do not identify useful regions, addressing can become the main cost.

Weight-space arithmetic is not invariant. Linear combinations of raw parameters are coordinate-dependent. Permutation symmetries, basin mismatch, sign conflicts, and incompatible representations can make naive averaging or task-vector arithmetic fail [Ainsworth et al., 2023, Yadav et al., 2023].

Deltas are path-dependent. A delta computed at parent θ_u is guaranteed to reconstruct its child only relative to that parent. It is not automatically a portable skill. Reusing deltas elsewhere requires alignment, transport, or empirical validation.

Lossless recall and aggressive consolidation conflict. Lossless refactoring preserves old states but saves only what exact shared structure permits. Aggressive compression improves memory and search cost but gives up the exact-recall guarantee.

Energy may not match task relevance. A parameter state that reconstructs x well may not be the state that predicts the desired variable well. The energy must include the variables and task constraints that actually matter.

12 Seed Research Plan

A minimal research program should test the perspective before adding speculative machinery.

Stage 1: Toy addressed-memory learner. Implement a small EBM or predictive-coding model with explicit parameter-delta storage:

$$\theta_{\text{eff}}(a) = \theta_0 + \sum_j a_j \Delta_j.$$

Compare online SGD, replay, exact leaf retention, and addressed inference on non-IID task streams. Measure current loss, old-task retention, memory growth, addressing cost, and graph depth.

Stage 2: Lossless versus lossy consolidation. Introduce regional averaging, low-rank delta bases, sparse projections, and residual dropping. Track the energy increase on retained examples:

$$\Delta \mathcal{E}_i = \min_z \mathcal{E}(x_i, z, \theta_{G'}(a'_i)) - \min_z \mathcal{E}(x_i, z, \theta_G(a_i)).$$

Compare weight-space reconstruction against data-space fidelity using replay or generative samples.

Stage 3: Amortized addressing. Add content-linked indices and learned routers that propose candidate graph regions. Measure the tradeoff between fast approximate recall and slow exact recall.

Stage 4: MoE and adapter specializations. Instantiate shallow routed modules, low-rank delta atoms, and adapter banks as controlled restrictions of the same graph. Compare against standard MoE, LoRA, and adapter baselines.

Stage 5: Columnar residual graft. Implement Goertzel-style sparse residual columns as macro-deltas. Test whether column routing improves retention and whether column-local delta DAGs add the stronger exact-recall property.

Stage 6: Metrics. Use RBII-style metrics: online performance, mean forgetting, sequential-versus-joint gap, transfer to held-out regimes, memory growth, description length, addressing cost, consolidation cost, and recovery latency. Add internal metrics from the residual literature: gradient conflict, off-support leakage, curvature coupling, commutator proxies, causal fingerprint separability, and column-template binding stability [Goertzel, 2026].

13 Conclusion

The addressed-parameter-memory view reframes continual learning as the resource-bounded management of a searchable, compressible history of parameter states. Ordinary SGD appears as the limiting case in which all addresses are collapsed into one global vector by immediate averaging. MoE appears as the limiting case in which a shallow learned router selects among parallel modules. Adapters, LoRA, masks, replay, EWC, GEM, OGD, task vectors, hypernetworks, model soups, and test-time adaptation are all interpretable as partial solutions to the same underlying problem: how to store, retrieve, protect, compose, and compress useful parameter changes.

The exact-recall formulation is useful because it makes forgetting mechanisms explicit. But the practical goal is not to store every state forever. The practical goal is to control the tradeoff among memory, compute, addressing cost, consolidation, and tolerated forgetting. Generalization emerges when compression discovers shared structure among historical parameter states.

Predictive coding is a convenient first instantiation because it naturally supports iterative inference over latent states and admits an extension to parameter-address variables. But the core perspective is broader: any energy-based or variational neural system that can infer both latent explanations and parameter addresses can implement the same idea.

Goertzel’s columnar residual architecture fits naturally inside this view. Its sparse columns are macro-deltas, its gates are amortized addresses, its rank-one atoms are delta bases, and its symbolic head is a content-addressing substrate. The additional symbolic machinery is not required for continual learning in the narrow sense, but it is a principled way to make the address space semantically stable and linkable to a symbolic reasoning system. To obtain the stronger exact-recall property, however, the columns themselves should be given local archival or delta-DAG structure rather than being treated as ordinary mutable experts.

References

- Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*, 2023.
- Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. Orthogonal gradient descent for continual learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*, 2020.
- Chrisantha Fernando, Dylan Banarse, Charles Blundell, et al. Pathnet: Evolution channels gradient descent in super neural networks. In *arXiv preprint arXiv:1701.08734*, 2017.

- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*, 2017.
- Ben Goertzel. A staged research plan for predictive-coding residual learning with symbolic heads, on top of frozen large language models, May 2026. Unpublished research plan.
- David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, et al. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, et al. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Yann LeCun, Sumit Chopra, Raia Hadsell, Marc’Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. In *Predicting Structured Data*, 2006.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017.
- Daniel MacDonald. Continual learning from engineering principles, 2026a. Unpublished AGI-26 manuscript.
- Daniel MacDonald. Resource-bounded incremental induction for continual learning, 2026b. Unpublished manuscript.
- Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *European Conference on Computer Vision*, 2018.
- Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2:79–87, 1999.
- Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, et al. Progressive neural networks. In *arXiv preprint arXiv:1606.04671*, 2016.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, 2017.
- James C. R. Whittington and Rafal Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local hebbian synaptic plasticity. *Neural Computation*, 29(5):1229–1262, 2017.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.

Prateek Yadav, Derek Tam, Leshem Choshen, et al. Ties-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, 2023.